

```
print("Hello, World!")
```



LINGUAGEM DE PROGRAMAÇÃO PYTHON



AULA 03

**SANTA
CRUZ**
CENTRO UNIVERSITÁRIO

ANTERIORMENTE

- **Fork, Branches, Push e Pull no Git Hub;**
- **Comentários e Docstrings;**
- **Conceito de comandos de entrada/saída (input e print);**
- **Operadores aritméticos e relacionais;**
- **Conversão de tipos (int, float, str);**

PRÓXIMOS PASSOS

- **Apresentação de strings;**
- **Concatenação, f-strings**
- **Métodos;**
- **Apresentação das condições lógicas;**
- **Onde utilizar estruturas condicionais;**
- **Estrutura condicional simples (if);**
- **Estrutura condicional composta (elif);**

Revisão de Conceitos do Git

- Sistema de controle de versão distribuído.
- Rastreia o histórico de alterações no código ao longo do tempo.
- **Repositório:** O local onde o projeto e seu histórico ficam armazenados.
- **Commit:** A ação de salvar uma nova versão do trabalho no GitHub com uma mensagem explicativa.
- **Branch:** Linha do tempo paralela para testar novas funcionalidades sem afetar o código principal.

Revisão de Conceitos do Git

- Fork: Copiar o repositório do professor/projeto para a sua conta.
- Pull Request: Solicitar a integração das suas alterações de volta ao projeto original.

Próximos passos:

- Review e Code Review.
- Merge.
- Sincronizar Fork.

GIT HUB

1. No Colab, clique em Arquivo > Salvar uma cópia no GitHub.
2. Na janela que abrir, chamado Caminho do arquivo (ou File path).
3. Digitar o nome da pasta com uma barra / antes do nome do arquivo.
 - Exemplo:
aula_03/taciany_lima_aula_03_ex01.ipynb

Atividade Prática 03:

Inserir **input()**, converter tipos, fazer cálculos e **comparações** com **operadores relacionais** no seu próprio personagem/usuário.

Crie pelo menos 3 novas variáveis com **operações aritméticas** e **comparações**, como também **conversões**.

Mostre o resultado dos seus cálculos e testes usando **print()**.

O que são Strings?

- **Definição:** Em Python, uma String (ou str) é um tipo de dado usado para representar textos, ou seja, uma sequência de caracteres (letras, números, símbolos).
- **Como criar:** Você pode definir uma string colocando o texto entre aspas simples (' ') ou aspas duplas (" "), ou ainda aspas triplas ('''')('''''''''').

Exemplos no código:

```
nome = "Gandalf"
```

```
local = 'Terra-Média'
```

```
ano = " 2019"
```

```
local_ano = ""
```

```
Gandalf está na Terra-Média.
```

```
Ano: 2019.
```

```
'''
```

Juntando Textos (Concatenação)

- **Definição:** Concatenação é o processo de unir duas ou mais strings para formar um único texto.
- **Como fazer:** Utilizamos o operador matemático de adição (+), com strings ele funciona "colando" as palavras.

Exemplo

```
nome = "Gandalf"
```

```
sobrenome = "O Branco"
```

```
# O espaço " " vazio é adicionado para separar  
as palavras
```

```
nome_completo = nome + " " + sobrenome
```

```
print(nome_completo)
```

```
# Resultado Esperado: Gandalf O Branco
```

Multiplicando Textos (Repetição)

- **Definição:** É o processo de repetir duas ou mais vezes uma palavra, caractere ou texto.
- **Como fazer:** Utilizamos o operador de multiplicação(*).
- Com strings ele funciona "repetindo" as palavras.

Exemplo

```
opcao_cardapio= "Spam "  
quantidade_spam=int(input("Quantos Spam você quer? "))  
total=opcao_cardapio*quantidade_spam  
print(total)
```

```
Quantos Spam você quer? 5  
Spam Spam Spam Spam Spam
```


f-strings

- **O que é:** As f-strings são a forma mais moderna, fácil e legível de misturar textos e variáveis no Python.
- **Como usar:** Basta colocar a letra **f** antes das aspas iniciais e colocar as variáveis dentro de chaves **{ }**.

f-strings

```
nome = "Aragorn, filho de Arathorn, herdeiro de  
Isildur"
```

```
idade = 87
```

```
# Misturando texto e números (variáveis) de forma  
simples
```

```
frase = f"Saudações, meu nome é {nome} e eu  
tenho {idade} anos."
```

```
print(frase)
```

```
# Resultado Esperado: Saudações, meu nome é  
Aragorn e eu tenho 87 anos.
```


Aspas dentro da string?

- Usar aspas simples dentro de uma string declarada com aspas duplas e vice-versa.
- Usar o caractere de escape \

Aspas dentro da string

```
# Sem o escape, isso gera um erro de sintaxe:  
# mensagem = "Ela disse: "Olá!" para todos."
```

```
# Com o caractere de escape \":  
mensagem = "Ela disse: \"Olá, Mundo!\" para  
todos."
```

```
print(mensagem)
```

```
# Saída: Ela disse: "Olá!" para todos.
```

[0]
[4]
[-2]

Sequência de caracteres

- Podemos acessar caracteres individuais usando índices.
- Índices iniciam sempre no zero.
- Índices podem ser negativos (“contagem de trás para frente”).

Sequência de caracteres

```
texto_strings = "Um Anel para a todos governar"
```

```
# Pegando a primeira letra
```

```
primeira_letra = texto_strings[0]
```

```
print(primeira_letra) # Saída: U
```

```
# Pegando a letra 'l' da segunda palavra
```

```
letra_seis = texto_strings[6]
```

```
print(letra_seis) # Saída: l
```

[0:]
[:4]
[:]

Recuperar substrings(Range)

No lugar de um índice podemos utilizar um range de índices.

Indicar o primeiro e último índice do range separados por:

- O primeiro é incluído na substring, o último não;
- Se o último índice não for indicado explicitamente, o Python irá fatiar até o último caractere da string;
- Se o primeiro índice não for indicado explicitamente, o Python irá fatiar a partir do primeiro caractere da string.

Recuperar substrings

```
texto_strings = "Um Anel para a todos governar"
primeira_palavra = texto_strings[:7]           # Do início até o índice 6
mensagem_principal = texto_strings[12:]        # Do índice 12 até o final
trecho_meio = texto_strings[7:12]              # Do índice 7 ao 11
trecho_sem_indice_definido = texto_strings[:]
print(f"Início: {primeira_palavra}")           # Saída: Um Anel
print(f"Fim: {mensagem_principal}")           # Saída: para a todos governar
print(f"Meio: {trecho_meio}")
print(f"Sem índice definido: {trecho_sem_indice_definido}")
```

Início: Um Anel

Fim: a todos governar

Meio: para

Sem índice definido: Um Anel para a todos governar

Funções para Tratamento de Strings

O que são: São ações prontas que o Python nos oferece para modificar ou transformar um dado. No caso das strings, eles ajudam a formatar o texto.

Como usar: Colocamos um ponto . logo após a variável de texto, seguido do nome do método e parênteses ().

Exemplo: Os usuários podem digitar textos com espaços sobrando sem querer ou misturar letras maiúsculas e minúsculas. Precisamos "limpar" isso para o programa entender.

Métodos

- **upper():** Transforma tudo em MAIÚSCULAS (ótimo para gritos!).
- **lower():** Transforma tudo em minúsculas (ótimo para sussurros).
- **capitalize():** Deixa só a primeira letra maiúscula.
- **title():** Deixa todas as letras de cada palavra maiúscula

Lower e Upper

```
grito_guerra = "por frodo!"
```

```
# O método .upper() transforma o texto para maiúsculas  
print(grito_guerra.upper())  
# Resultado Esperado: POR FRODO!
```

```
feitico = "MELLON" # (Amigo em élfico)
```

```
# O método .lower() transforma o texto para minúsculas  
print(feitico.lower())  
# Resultado Esperado: mellon
```

Capitalize e Inversão

```
titulo = "o Senhor dos aneis"
```

```
# Capitalizando (primeira letra maiúscula)
```

```
print(titulo.capitalize())
```

```
# Saída: O senhor dos aneis
```

```
# Invertendo o texto usando fatiamento com passo negativo[::-1]
```

```
inscricao = "Um Anel para a todos governar"
```

```
texto_invertido = inscricao[::-1]
```

```
print(f"Inscrição ao contrário: {texto_invertido}")
```

```
# Saída: ranrevog sodot a arap lenA mU
```

Funções para Tratamento de Strings

- **len():** Função utilizada para determinar o tamanho de uma string (tamanho determinado em quantidade de caracteres);
- Os espaços também são quantificados.

len()

▶ # A função len() conta a quantidade total de caracteres, incluindo espaços!

Exemplo 1: O nome de um personagem

mag0 = "Gandalf, o Cinzento"

tamanho_nome = len(mag0)

print(f"O nome {mag0}' possui {tamanho_nome} caracteres.")

Saída: O nome 'Gandalf, o Cinzento' possui 19 caracteres.

O nome 'Gandalf, o Cinzento' possui 19 caracteres.

Funções para Tratamento de Strings

Replace:

- É utilizado para substituir uma string por outra.
- A string substituída e sua substituta são informadas como parâmetros do método.
- Sintaxe: `string.replace("texto_antigo", "texto_novo")`
- Importante: O método `.replace()` não altera a variável original, ele cria e retorna uma nova string com as substituições feitas.

Replace

```
# Texto original da profecia
texto_profecia = '''Um Anel para a todos governar, Um Anel para encontrá-los,
Um Anel para a todos trazer e na escuridão aprisioná-los na terra de Mordor.'''
local_profecia = input("Digite o local da profecia: ")
# Substituindo 'Mordor' por 'Gondor' ou modificando o nome do reino
nova_profecia = texto_profecia.replace("Mordor", local_profecia)

print("Original:")
print(texto_profecia)
|
print("\nCom Replace:")
print(nova_profecia)
```

Funções para Tratamento de Strings

Split:

- A string original é separada com base em um “separador” passado como parâmetro no método.
- **Padrão:** Se você não passar nenhum parâmetro `string.split()`, ele divide o texto em todos os espaços em branco.
- **Resultado:** Ele sempre retorna uma lista (list) com os pedaços da string original.
- Estudaremos listas daqui a algumas aulas. Agora, vamos utilizar somente o índice para acessar seus itens, como nos caracteres das strings.

Split

```
# String recebida com os nomes separados por vírgula
lista_membros = "Frodo, Sam, Aragorn, Legolas, Gimli, Boromir, Gandalf"

# O split() divide a string a cada vírgula e cria uma lista de membros
membros_sociedade = lista_membros.split(", ")

print("Escolhido:")
print(f"- {membros_sociedade[2]}")
```

```
Escolhido:
- Aragorn
```


Funções para Tratamento de Strings

Find():

Usado para buscar uma substring.

- Sintaxe: `string.find("texto_procurado")`
- **Retorno de Sucesso:** Retorna o índice (posição numérica) onde a primeira ocorrência do texto começa (lembrando que em Python a contagem inicia em 0).
- **Quando não encontra:** Retorna -1 (diferente de outros métodos que geram erro no programa, o `.find()` é ótimo para checagens simples sem interromper o código).

Find

```
# Registro da ordem de chegada para pegar os livros sagrados
registro_biblioteca = "Presentes: Merry, Pippin, Frodo, Sam, Boromir"

# Procurando a posição de 'Frodo'
posicao_frodo = registro_biblioteca.find("Frodo")
print(f"Posição do primeiro caractere de 'Frodo': {posicao_frodo}")
```

Posição do primeiro caractere de 'Frodo': 26

Funções para Tratamento de Strings

Count:

Conta o número de vezes que uma substring está contida na String.

- Sintaxe: `string.count("texto_procurado")`
- Diferencia maiúsculas e minúsculas (case-sensitive): "A" e "a" são contados separadamente.
- Funciona em Listas: Diferente do `.find()` e do `.split()`

Count

```
# Histórico de presenças da semana gravado em um texto
historico_presenca = "Frodo, Sam, Frodo, Legolas, Frodo, Gimli, Sam"

# O count conta quantas vezes a palavra 'Frodo' aparece no texto
aulas_presente = historico_presenca.count("Frodo")

print("Dias em que Frodo esteve presente:", aulas_presente)

Dias em que Frodo esteve presente: 3
```

Funções para Tratamento de Strings

- **strip():**
- É utilizada para remover caracteres indesejados do início e do final de strings:
- Se o caractere não for informado como parâmetro, removerá os espaços.
- É possível remover apenas do início ou do final usando lstrip e rstrip respectivamente.

Atividade Prática 01 Aula 03

Análise de Mutação no Gene HBB (Hemoglobina)

Contexto da Atividade: A Beta-Hemoglobina é uma proteína vital para o transporte de oxigênio no sangue. Uma alteração em apenas um nucleotídeo do gene HBB pode resultar em Anemia Falciforme.

Seu **objetivo** nesta atividade é construir um script em Python no Google Colab que processe uma fita de DNA, faça a tradução para aminoácidos e diagnostique se a amostra pertence a uma hemoglobina normal ou mutante.

Atividade Prática 01 Aula 03

Análise de Mutação no Gene HBB (Hemoglobina)

****Entrada:**** Sequência de DNA do gene HBB (atggtgcacctgactcctgag).

Solicite que o usuário digite uma fita de DNA com 21 nucleotídeos através da função input().

****Processamento 1:**** Padronização para maiúsculas e formatação de texto (.upper(), *, f-strings).
Exiba a fita formatada usando f-string.

Atividade Prática 01 Aula 03

Análise de Mutação no Gene HBB (Hemoglobina)

Traduza a fita de DNA substituindo as trincas (códon) pelos seus respectivos aminoácidos utilizando o método .replace() encadeado.

Tabela de Códon do Gene HBB:

- ATG=Met-
- GTG=Val-
- CAC= His-
- CTG= Leu-
- ACT= Thr-
- CCT= Pro-
- GAG= Glu-

Atividade Prática 01 Aula 03

Análise de Mutação no Gene HBB (Hemoglobina)

Defina a sequência de referência normal da Hemoglobina A:
"ATGGTGCACCTGACTCCTGAG".

Compare a fita digitada pelo usuário com a fita de referência usando o operador de diferença (!=) para obter um valor Booleano (True se for mutante, False se for normal).

Crie uma lista com as duas opções de diagnóstico:

[" Sequência NORMAL (Hemoglobina A)", " Sequência MUTANTE (Anemia Falciforme - HbS)"]

Atividade Prática 01 Aula 03

Análise de Mutação no Gene HBB (Hemoglobina)

Exiba o resultado acessando o índice da lista com a variável booleana (`diagnosticos[mutante]`), sem utilizar a instrução `if`.

Solicite ao usuário um aminoácido para busca (ex: Val ou His) e localize seu índice na proteína traduzida usando `.find()`.

Imprima o relatório final contendo: DNA de referência, DNA digitado, tamanho da fita (`len()`), proteína traduzida, resultado do diagnóstico e a posição do aminoácido pesquisado.

strip()

A senha secreta da porta está toda em letras minúsculas e sem espaços

```
senha_secreta = "mellon"
```

O jogador digita a senha no teclado (com espaços sobrando

```
entrada_jogador = "mellon "
```

```
print(senha_secreta == entrada_jogador)
```

Sem limpar os dados, a porta continuaria fechada, pois "mellon " é diferente de "mellon".

```
senha_tratada = entrada_jogador.strip()
```

```
print(senha_secreta == senha_tratada)
```

Funções para Tratamento de Strings

- **ord() e chr():**
- A função ord() (Letra-> Número)
- Pega uma única letra e devolve o número de identificação dela na tabela do computador.

- A função chr() (Número → Letra)
- Faz exatamente o caminho inverso. Você entrega um número e ela devolve qual caractere ele representa.

ord() e chr()

```
numero_do_A = ord("A")  
print(numero_do_A)  
# Resultado: 65
```

```
numero_do_B = ord("B")  
print(numero_do_B)  
# Resultado: 66
```

```
letra_65 = chr(65)  
print(letra_65)  
# Resultado: A  
  
letra_97 = chr(97)  
print(letra_97)  
# Resultado: a (letras minúsculas têm números diferentes!)
```

A
a

`.strip()`
`.lower()`
`.split()`

Funções para Tratamento de Strings

Combos (Encadeamento): No Python, podemos usar mais de um método ao mesmo tempo para tratar o dado de uma vez só!

- Código mais limpo: Evita criar variáveis intermediárias desnecessárias.
- Ordem de leitura: A execução é feita da esquerda para a direita.

Regra de ouro: O método da esquerda precisa retornar um tipo de dado compatível com o método da direita (ex: **.strip()** retorna uma **string**, então podemos chamar **.lower()** ou **.split()** em seguida)

Encadeamento

```
# A senha secreta está toda em letras minúsculas e sem espaços
senha_secreta = "mellon"
# O jogador digita a senha no teclado (com espaços sobrando e letras maiúsculas)
entrada_jogador = "MELLON "
print("Sem tratamento:", senha_secreta == entrada_jogador)
# Sem limpar os dados, a porta continuaria fechada
senha_tratada_parcial = entrada_jogador.strip()
print("Com tratamento parcial:", senha_secreta == senha_tratada_parcial)
# False ainda pois "MELLON" é diferente de "mellon".
# Vamos encadear o .strip() e o .lower() para tratar essa entrada:
senha_tratada = entrada_jogador.strip().lower()
print("Com tratamento completo", senha_secreta == senha_tratada)
```

Sem tratamento: False

Com tratamento parcial: False

Com tratamento completo True

`::xf`

`::,`

`::x%`

Formatando números

- Para escolher a quantidade de casas decimais podemos utilizar a sintaxe **`::xf`**, onde **x** é o número de casas decimais.
- Para adicionar separador de milhares (,) em números grandes utilizamos a sintaxe **`::,`**
- Para transformar um número em porcentagem, podemos utilizar a sintaxe **`::x%`**, onde **x** é o número de casas decimais!

Formatando números

```
nota1 = 8.75
```

```
nota2 = 9.30
```

```
nota3 = 7.80
```

```
# Cálculo da média simples
```

```
media = (nota1 + nota2 + nota3) / 3
```

```
print(f"A média do aluno Samwise é: {media}")
```

```
# Uso do :.2f dentro da f-string para limitar as casas decimais
```

```
print(f"A média do aluno Samwise é: {media:.2f}")
```

```
A média do aluno Samwise é: 8.616666666666667
```

```
A média do aluno Samwise é: 8.62
```

Formatando números

```
ouro_arrecadado = 1550000
```

```
# O uso de :, adiciona vírgula como separador de milhar (padrão norte-americano/python)  
print(f"Total arrecadado para a biblioteca: {ouro_arrecadado:,} moedas de ouro.")
```

```
# Ou substituindo por ponto (padrão brasileiro)  
formatado_br = f"{ouro_arrecadado:,}".replace(",", ".", ".")  
print(f"Formatado em PT-BR: R$ {formatado_br}")
```

```
Total arrecadado para a biblioteca: 1,550,000 moedas de ouro.  
Formatado em PT-BR: R$ 1.550.000
```

Formatando números

```
# Acertos no teste: 18 de 23 questões  
taxa_acerto = 18 / 23 # Dá aproximadamente 0.782608...
```

```
# 0 :.2% formata com 2 casas decimais e o símbolo de %  
print(f"Taxa de acerto do Frodo: {taxa_acerto:.2%}")
```

Taxa de acerto do Frodo: 78.26%

Atividade Prática 02 Aula 03

Objetivo: Utilizar variáveis, métodos de manipulação de strings e índices para limpar, analisar e extrair informações de um texto de forma sequencial.

A Missão: A Sociedade do Anel encontrou um pergaminho antigo, mas o texto está bagunçado! Seu trabalho é criar um programa em Python que receba esse texto, faça a higienização dos dados e gere um relatório.

Passo a Passo (Requisitos do Sistema):

****Entrada de Dados:****

- O programa deve solicitar que o usuário digite o texto do pergaminho.
- Na frase digitada ou copiada incluía espaços a mais no começo e fim.

****Limpeza e Higienização**:**

- Remova os espaços inúteis no início e no final do texto digitado e guarde em uma variável.

Atividade Prática 02 Aula 03

****O Relatório****

- O programa deve calcular e exibir na tela:
- O número total de caracteres do texto limpo.* A quantidade total de palavras presentes no texto.
- A primeira palavra do texto.
- A última palavra do texto.

****Detector de Ameaças**:**

- O sistema deve verificar se a palavra, p.e. "Anel", ou outra palavra a sua escolha, existe dentro do texto e imprimir o resultado diretamente;
- O sistema deve contar quantas vezes a palavra aparece.

Próximos Passos



**Condições lógicas;
Onde utilizar estruturas condicionais;
Estrutura condicional simples (if);
Estrutura condicional composta (elif e else);**

OBRIGADA!

“Complex is better than
complicated.”

The Zen of Python, by Tim Peters



SANTA CRUZ

CENTRO UNIVERSITÁRIO

Para mais
informações
acesso
o **QR CODE**
e saiba mais



 (41)3052-4900

 @unisantacruz

 /UniSantaCruzCtba

 unisantacruztem.com.br